What about a database?

Git and an XML database can work well together. What matters is which tasks they perform and where the authoritative edition data and their history are held.

Your IT team proposes an XML database. Do you still need Git? And does choosing Git mean giving up powerful queries? This resource explains how the tasks fit together. It is not an introduction to a particular database product.

> The Albrecht team needs to find every letter with an uncertain date and a reference to a particular person. This question comes up repeatedly. A shared XML query service can make the task easier. Samir also asks: which data version did the query use—and where will we record the resulting corrections?

Tasks

Record changes

Git versions files and related states. It can show how TEI, a schema and processing software changed together.

Example and tip

A corrected date and the corresponding change to a checking rule are recorded together.

Investigate XML

XML tools and XML databases interpret encoded structures. XPath and XQuery help with reference, checking and analysis. Git does not run these queries itself.

Example and tip

A query finds letters with uncertain dates that refer to the same person.

Produce publications

Transformations and applications create views, indexes and search facilities. A technical frontend provides an interface for exploring the edition.

Example and tip

An approved TEI version supplies web views and indexes. The version used is documented.

Preserve results

Versioned data, publication outputs and production resources are linked and independently preserved. A running service and an archival copy serve different purposes.

Example and tip

A release identifies the data, programs and web output; the archive receives the agreed package.

What does a database actually add?

An XML database such as eXist-db can provide a shared, indexed collection for repeated structured queries and applications. This helps when combinations change frequently, many people investigate the same collection, or querying files is no longer practical.

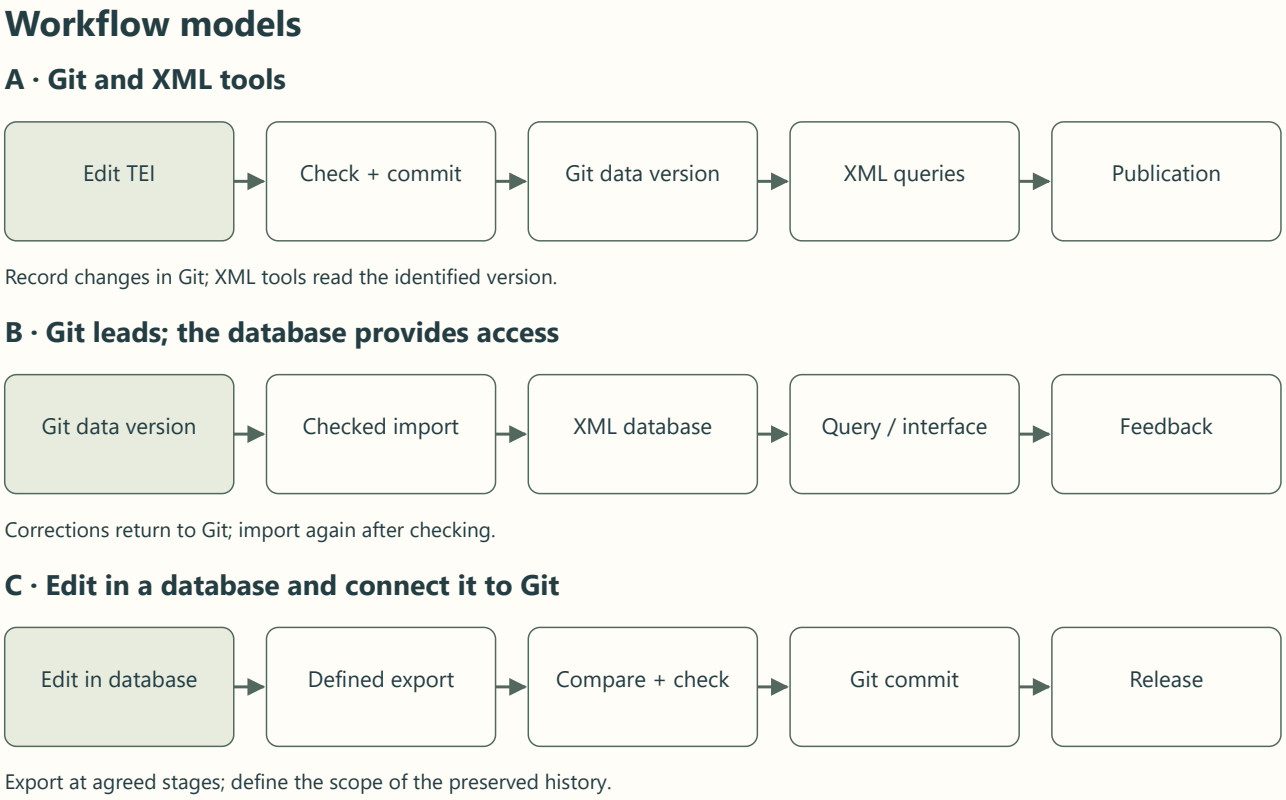
File-based XML tools, such as oXygen or Saxon, may also meet the same editorial need. XPath expresses paths and conditions in XML; XQuery can select and combine information from XML documents. An index maintains prepared search structures that can speed up suitable queries. Both approaches require access to the necessary XML expertise.

Next step

Try three real editorial queries and two intended web functions on your actual collection. Compare results, working time, setup and maintenance. Neither a fixed file count nor team size determines whether you need a database.

How we would begin

We organise TEI files, the model, queries, transformations and documentation as a traceable collection in Git. XML tools provide query capabilities. We add a database service when it brings a concrete benefit. An existing, supported institutional service can make sense even for a small project.



Schematic diagram · Workflow models

Read the diagram as text

1. **Git and XML tools**: Edit TEI → Check + commit → Git data version → XML queries → Publication. Git holds the authoritative collection. Editors compare and check changes, record them as commits and transfer them to the shared repository. XML tools read the files. Publication can be generated statically; a running application is also possible.
2. **Git leads; the database provides access**: Git data version → Checked import → XML database → Query / interface → Feedback. Git holds the authoritative edition data. A checked import builds the database collection. The database serves queries and the interface. Editorial corrections return to Git through the agreed review process; the database is then updated. There is no uncontrolled second place for edits.

3. **Edit in a database and connect it to Git**: Edit in database → Defined export → Compare + check → Git commit → Release. An existing database interface remains the authoritative editing environment. A defined export passes checked changes to Git, which versions the exported data and associated programs. Editing exported files and the database simultaneously requires an agreed reconciliation process.

A · Git and XML tools

Git holds the authoritative collection. Editors compare and check changes, record them as commits and transfer them to the shared repository. XML tools read the files. Publication can be generated statically; a running application is also possible.

Example and tip

Example: Anna needs a list of uncertain dates. A documented XML query over her files is sufficient. She versions the query together with the edition data.

Tip: Begin with a repeatable query. Record the input version, tool and invocation. Then establish whether a central service adds useful capabilities.

B · Git leads; the database provides access

Git holds the authoritative edition data. A checked import builds the database collection. The database serves queries and the interface. Editorial corrections return to Git through the agreed review process; the database is then updated. There is no uncontrolled second place for edits.

Example and tip

Example: The Albrecht team investigates letter dates in the database. A finding leads to a correction in Git. The next checked import transfers that change to the service.

Tip: Link the data version, import script, application and index configuration in the deployment record. Rebuild an empty test installation from them. Explicitly capture and preserve any scholarly annotation created only in the database.

C · Edit in a database and connect it to Git

An existing database interface remains the authoritative editing environment. A defined export passes checked changes to Git, which versions the exported data and associated programs. Editing exported files and the database simultaneously requires an agreed reconciliation process.

Example and tip

Example: The team already works with an institutional XML service. After an agreed editing step, it exports documents, checks deletions, references and completeness, and records the change in Git.

Tip: Specify export triggers, scope, responsible roles and error handling. A nightly export preserves daily states, but does not automatically capture every intermediate change, attribution or decision rationale.

How do we access the data?

Checked-out TEI files can be opened directly with a suitable tool. Git makes earlier file versions available. Regular access to a database's internal collection uses the database software, for example through XQuery, an API or export. Exported XML is independently usable too. Check which information and relationships the export actually includes.

Authoritative collection

The agreed location whose approved contents govern transfer and publication. Other copies must not diverge unnoticed.

Build

A documented processing run that generates outputs such as HTML, indexes and search structures from data and rules.

Release

A deliberately approved, identified version. Git tags mark the associated commit; archiving is a separate step.

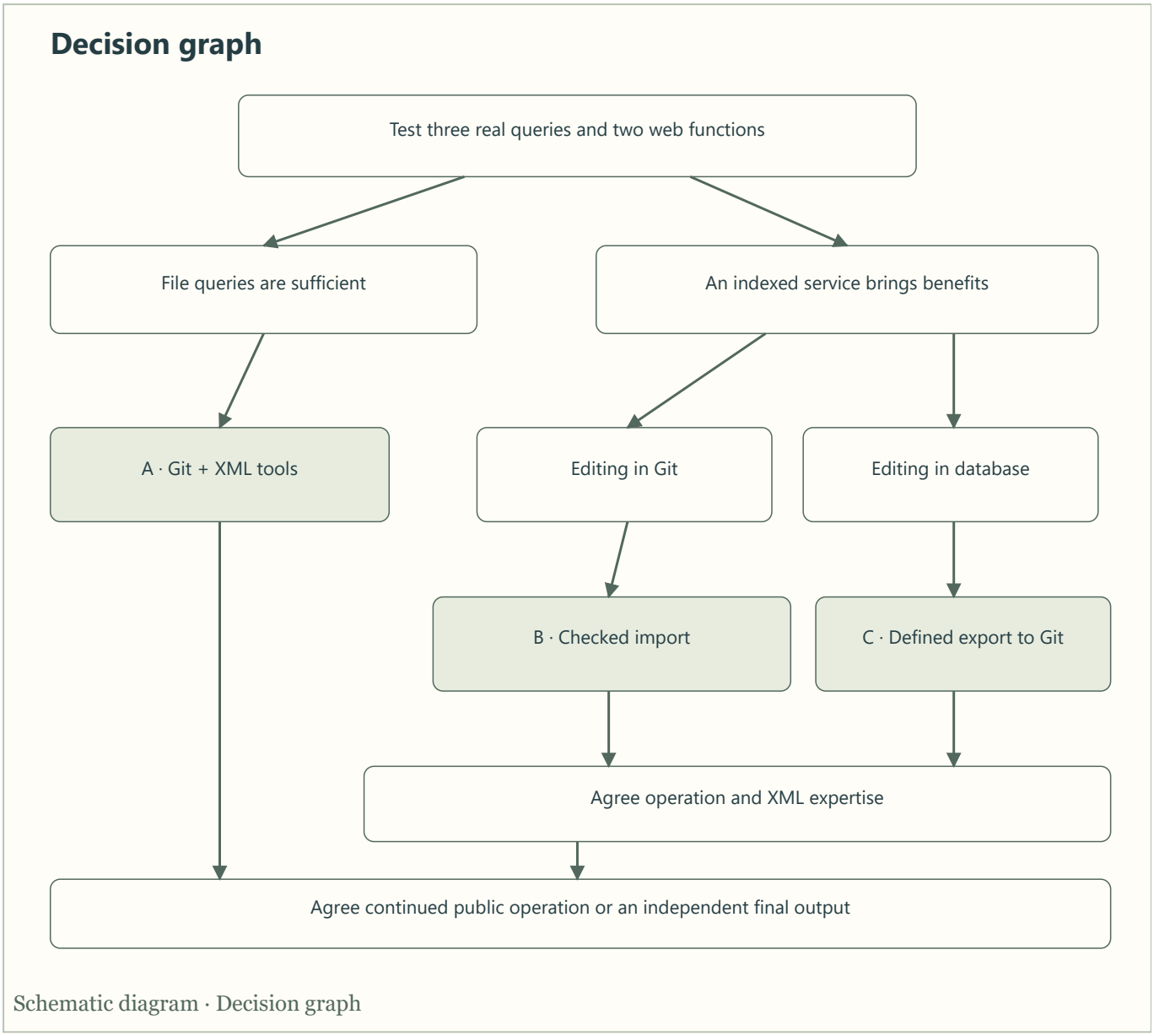
Why we preserve the history

A later editorial team asks why certain dates were once considered uncertain. The preserved history reveals the earlier markup, its revision and the related checking rule. Commit messages or decision notes add the reasons. Editorial experiments and insights remain open to investigation, alongside final results.

A database solution can also be configured with versioning. This does not automatically create a connected history of changes to texts, schemas and programs. We recommend preserving that history with Git and using a database for its concrete benefits.

> Next step
>
> Preserve the relevant complete Git history, including models, programs and decision documents. Agree separately who may read it later: preservation does not automatically mean public release.

Do we need a database service?



Schematic diagram · Decision graph

Read the diagram as text

1. Which queries do we need?: Not yet tested in practice / File-based queries are sufficient / A shared indexed service brings a concrete benefit
2. Where are the edition data edited?: In files, using Git / In an existing database interface
3. Who will support a database service?: Not yet agreed / A named service with an agreed scope
4. Must it remain publicly operational after the project?: No; an identified publication output is sufficient / Yes, for a necessary dynamic function

Our guidance: Test the need first

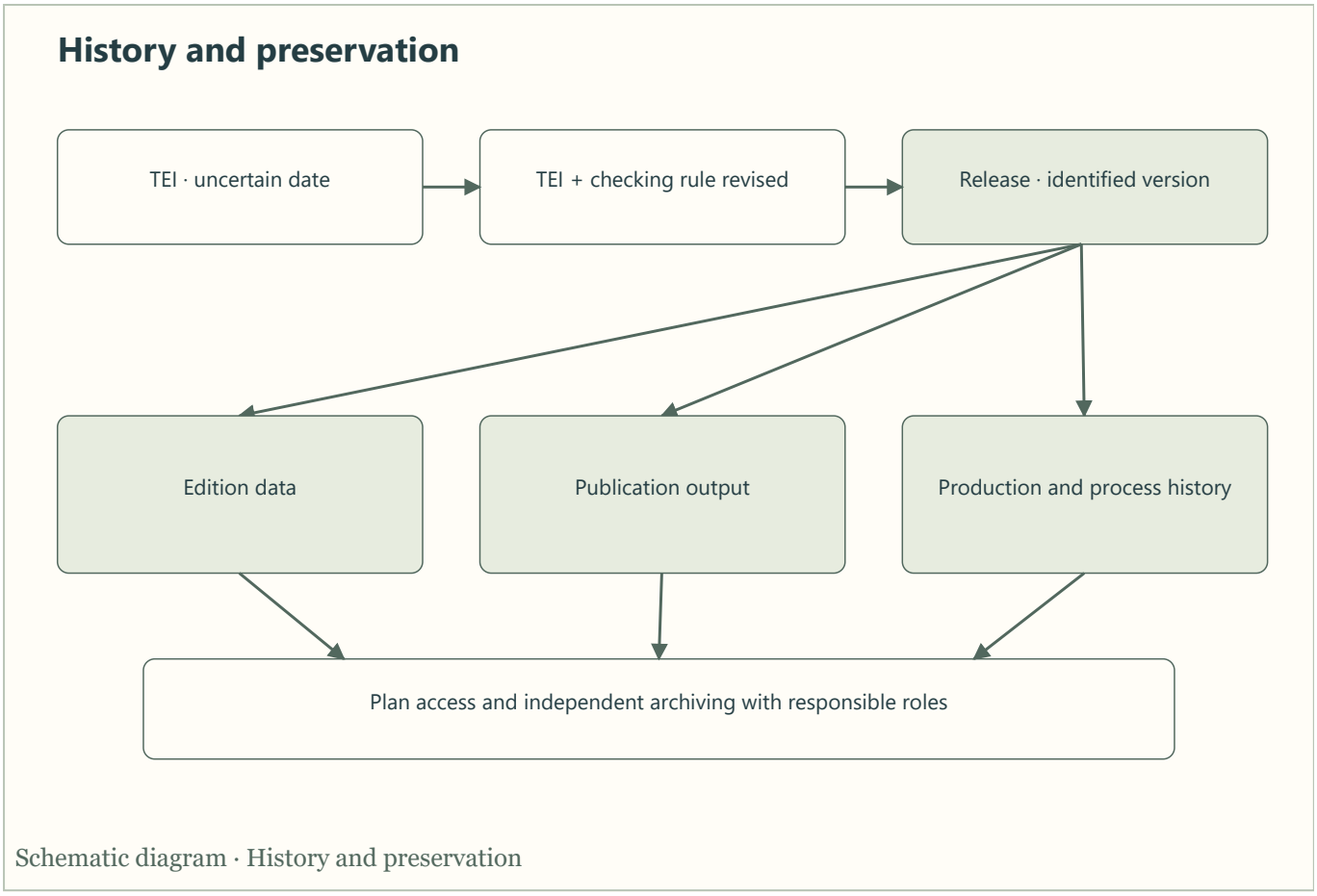
Begin with Git and a documented XML query. Test three real queries and two web functions before committing to an additional service.

> Before adding a database service, agree responsibility, available XML expertise and scope. A possible installation is not an operational commitment.

Check whether you can provide an independently usable publication output. A database can support editing without remaining part of the public edition indefinitely.

What remains after the project?

Three collections are linked to the same identified version. Plan public access and independent archiving separately, with responsible roles for each.



Schematic diagram · History and preservation

Read the diagram as text

1. **Edition data**: TEI, metadata, links, model, documentation and rights.
2. **Publication output**: The actual web edition, indexes, search index and any print/PDF output. Identify indispensable external services.
3. **Production and process history**: Relevant Git history, queries, transformations, tests, configurations, dependencies and decision notes.

Edition data

TEI, metadata, links, model, documentation and rights.

Publication output

The actual web edition, indexes, search index and any print/PDF output. Identify indispensable external services.

Production and process history

Relevant Git history, queries, transformations, tests, configurations, dependencies and decision notes.

Remember

Static or dynamic is a separate decision. Git can supply a running application; a database can generate static outputs. A Git repository replaces neither an archive service nor funding for a running system.

For the IT meeting and proposal

Your answers stay in this tab's session storage. Language switching retains them. If storage is blocked, you can still use and export the worksheet.

Three real XML queries and their purposes

Two required web functions; what can be precomputed?

Authoritative collection and permitted editing locations

Import/export: trigger, scope, checks and error handling

Which intermediate states and decision reasons will we preserve?

Link data, application, configuration and publication versions

Responsible roles, XML expertise and agreed services

Completion, continued operation and archiving: who does what?

Next practical test and allocated budget

Adaptable wording for a proposal

Git-based foundation

We maintain the edition data, documented model and production resources in Git. Publications are linked to identified data and tool versions. [Role/budget] is allocated to quality checks, completion and institutional handover.

Additional database service

We use [service] for [specific queries/function]. The authoritative collection, change process and rebuilding procedure are documented. [Responsible unit] provides [scope/duration]; continued operation or a completion alternative and archival deposit are agreed.

Continue in the strategy workshop →

Explore XML databases and tools further

This resource focuses on Git strategy. The official sources explain eXist-db and XML tools in greater depth, including community resources and professional support. Check product editions and service commitments for your particular project.

- eXist-db · Documentation
- eXist-db · XQuery

- eXist-db · Community / professional support
- Saxonica · Using XQuery
- Git · Documentation
- TEI · Guidelines

Editorial basis: the two supplied working documents on Git and XML databases. Text, translation, schematic diagrams and implementation were created with AI assistance. Version 1.0 · 28 October 2026. Diagrams show workflow models, not an actual installation.